

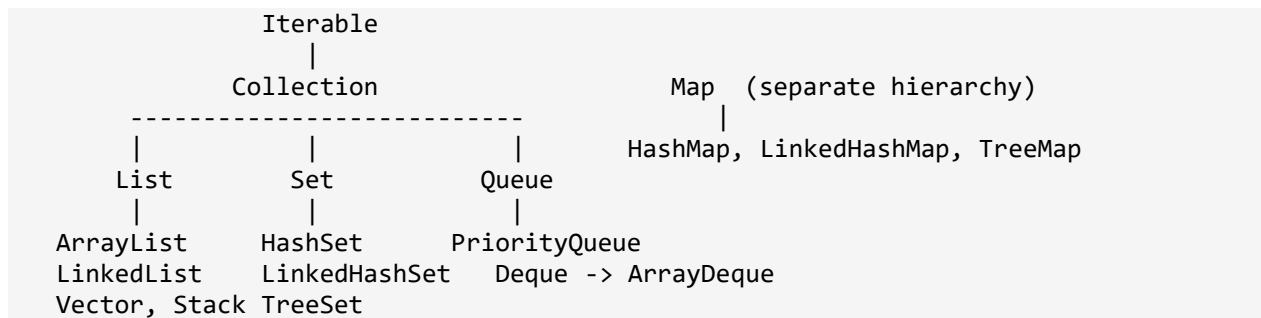
Java Interview Questions and Answers(part 2): Collection Framework and Strings

Section A has 35 questions on the Java Collection Framework. Section B has 15 questions on Strings. Every answer is in simple words, and the examples are in code blocks so they are easy to copy.

A. Java Collection Framework (35 Questions)

1. What is the Java Collection Framework?

The Java Collection Framework is a ready-made set of **interfaces, classes and algorithms** in the `java.util` package. It is used to store, search, sort and manage a **group of objects**. Instead of writing your own list or set, you pick a ready-made one such as `ArrayList`, `HashSet` or `HashMap`.



2. Why do we use collections instead of arrays?

Array	Collection
Fixed size, decided at creation	Grows and shrinks automatically
No ready-made methods	Ready-made methods: add, remove, search, sort
Only one data structure (a block of memory)	Many structures: list, set, queue, map
Can hold primitives directly (int, char)	Holds objects only (primitives are auto-boxed)

Collections also use **generics** (for example `List<String>`), which gives type safety.

3. What is the difference between Collection and Collections?

`Collection` is an **interface**, the root of `List`, `Set` and `Queue`. `Collections` is a **utility class** with static helper methods such as `sort()`, `reverse()`, `max()`, `min()` and `shuffle()`.

```
List<Integer> nums = new ArrayList<>(List.of(30, 10, 20));
Collections.sort(nums);                // [10, 20, 30]
Collections.reverse(nums);             // [30, 20, 10]
System.out.println(Collections.max(nums)); // 30
System.out.println(Collections.min(nums));  // 10
```

4. What is the difference between Collection and Map?

Collection	Map
Stores single elements	Stores key-value pairs
Add using <code>add(e)</code>	Add using <code>put(key, value)</code>
Can hold duplicates (List, Queue)	Keys are unique, values may repeat
Extends Iterable	Does not extend Collection

5. What is the difference between List, Set, and Queue?

	List	Set	Queue
Order	Insertion order, indexed	HashSet: none, LinkedHashSet: insertion, TreeSet: sorted	FIFO (PriorityQueue: by priority)
Duplicates	Allowed	Not allowed	Allowed
Access by index	Yes	No	No (only the head)
Examples	ArrayList, LinkedList	HashSet, TreeSet	ArrayDeque, PriorityQueue

6. What is the difference between ArrayList and LinkedList?

	ArrayList	LinkedList
Internal structure	Resizable array	Doubly linked list of nodes
<code>get(i)</code>	Fast, $O(1)$	Slow, $O(n)$
Add or remove at the end	Fast	Fast
Add or remove at the start	Slow (elements shift)	Fast
Add or remove in the middle	Elements shift	No shifting, but finding the position is $O(n)$
Memory	Less	More (each node stores two links)
Interfaces	List	List and Deque
Best for	Reading by index	Frequent insert/delete at both ends

7. What are the important features of ArrayList?

1. Resizable: grows automatically (default capacity 10, grows by about 50%).
2. Keeps insertion order.
3. Allows duplicate elements.
4. Allows null values.
5. Gives fast access by index.
6. Not synchronized (not thread-safe).

8. How do you add elements to an ArrayList?

```
List<String> fruits = new ArrayList<>();
fruits.add("Apple");           // [Apple]
fruits.add("Banana");          // [Apple, Banana]
fruits.add(1, "Mango");        // [Apple, Mango, Banana]
fruits.addAll(Arrays.asList("Orange", "Grapes")); // adds many at once
```

9. What is the difference between add() and add(index, element)?

`add(element)` puts the element at the **end** of the list. `add(index, element)` **inserts** it at the given position and shifts the existing elements to the right. The index must be from 0 to `size()`, otherwise `IndexOutOfBoundsException` is thrown.

```
List<String> list = new ArrayList<>(List.of("A", "B"));
list.add("C");      // [A, B, C]
list.add(1, "X");   // [A, X, B, C]
```

10. How do you access an element from an ArrayList?

Use `get(index)`. The index starts at 0. A wrong index throws `IndexOutOfBoundsException`.

```
List<String> fruits = List.of("Apple", "Banana", "Mango");
System.out.println(fruits.get(0)); // Apple
System.out.println(fruits.get(2)); // Mango
```

11. How do you remove an element from an ArrayList?

```
List<String> fruits = new ArrayList<>(List.of("Apple", "Banana", "Mango", "Orange"));
fruits.remove("Banana");           // by value
fruits.remove(0);                  // by index -> [Mango, Orange]
fruits.removeIf(f -> f.startsWith("O")); // by condition -> [Mango]
fruits.clear();                    // removes everything
```

12. What is the difference between remove(int index) and remove(Object)?

`remove(int index)` removes the element at that **position** and returns the removed element.
`remove(Object)` removes the **first matching value** and returns `true` or `false`. The confusion happens with `List<Integer>`.

```
List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 1));
nums.remove(1);           // removes INDEX 1 (value 20) -> [10, 30, 1]
nums.remove(Integer.valueOf(1)); // removes VALUE 1 -> [10, 30]
```

13. How do you find the size of an ArrayList?

Use `size()`. It gives the number of elements, not the internal capacity.

```
List<String> fruits = new ArrayList<>(List.of("Apple", "Banana"));
System.out.println(fruits.size());    // 2
System.out.println(fruits.isEmpty()); // false
```

14. How do you check whether an ArrayList contains a particular element?

Use `contains()`. It compares using `equals()`. `indexOf()` returns the position, or `-1` if absent.

```
List<String> fruits = List.of("Apple", "Banana");
System.out.println(fruits.contains("Apple"));    // true
System.out.println(fruits.indexOf("Banana"));    // 1
System.out.println(fruits.indexOf("Grapes"));    // -1
```

15. How do you iterate through an ArrayList?

```
List<String> fruits = new ArrayList<>(List.of("Apple", "Banana", "Mango"));

// 1. Normal for loop
for (int i = 0; i < fruits.size(); i++) {
    System.out.println(fruits.get(i));
}
// 2. For-each loop
for (String f : fruits) {
    System.out.println(f);
}
// 3. Iterator
Iterator<String> it = fruits.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
// 4. forEach with lambda
fruits.forEach(f -> System.out.println(f));
Do not add or remove elements inside a for-each loop. It throws ConcurrentModificationException.
Use an Iterator for removal.
```

16. What is a Set?

A Set is a collection that stores **unique elements only**. It has no index, so you cannot use `get(i)`. The main classes are `HashSet`, `LinkedHashSet` and `TreeSet`. `add()` returns `false` if the element is already present.

```
Set<String> fruits = new HashSet<>();
System.out.println(fruits.add("Apple"));    // true
System.out.println(fruits.add("Apple"));    // false (duplicate ignored)
```

17. What is the main difference between List and Set?

A **List** allows **duplicates** and keeps positions (index access). A **Set** **does not allow duplicates** and has no index. Use a List when order and repeats matter. Use a Set when you need only unique values.

18. Does a HashSet allow duplicate elements?

No. If you add a duplicate, it is ignored and `add()` returns `false`. `HashSet` decides what a duplicate is using `hashCode()` and `equals()`. It allows one null.

19. Does a HashSet maintain insertion order?

No. The order is not guaranteed and can look random. If you need insertion order, use `LinkedHashSet`. If you need sorted order, use `TreeSet`.

20. What is the difference between HashSet and LinkedHashSet?

Both reject duplicates and are fast ($O(1)$). `HashSet` has **no order**. `LinkedHashSet` keeps **insertion order** by also maintaining a linked list, so it uses a little more memory.

```
Set<String> linked = new LinkedHashSet<>();
linked.add("Banana");
linked.add("Apple");
linked.add("Mango");
System.out.println(linked);    // [Banana, Apple, Mango] (insertion order)
```

21. What is the difference between HashSet and TreeSet?

	HashSet	TreeSet
Order	None	Sorted (ascending by default)
Speed	$O(1)$	$O(\log n)$
Null	One null allowed	Null not allowed (<code>NullPointerException</code>)
Uses	<code>hashCode()</code> and <code>equals()</code>	<code>compareTo()</code> or a <code>Comparator</code>
Extra methods	None	<code>first()</code> , <code>last()</code> , <code>floor()</code> , <code>ceiling()</code> , <code>headSet()</code>

22. What is the purpose of TreeSet?

`TreeSet` stores **unique elements in sorted order**. It is useful when you need sorted unique data or range and neighbour queries, such as "smallest value", "largest value" or "next higher value".

```
TreeSet<Integer> numbers = new TreeSet<>(List.of(30, 10, 20));
System.out.println(numbers);           // [10, 20, 30]
System.out.println(numbers.first());   // 10
System.out.println(numbers.last());    // 30
System.out.println(numbers.higher(20)); // 30
```

23. How does TreeSet maintain its elements?

`TreeSet` is backed by a `TreeMap`, which uses a **Red-Black tree** (a self-balancing binary search tree). Each new element is placed by comparing it with existing ones, using `compareTo()` (`Comparable`) or a `Comparator`. Two elements are treated as duplicates when the comparison returns 0. This keeps the data sorted, and add, remove and search take $O(\log n)$.

```
TreeSet<String> names = new TreeSet<>(Comparator.reverseOrder());
names.add("Anu");
names.add("Chitra");
```

```
names.add("Bala");
System.out.println(names);    // [Chitra, Bala, Anu]
```

24. What is a Queue?

A Queue is a collection that follows **FIFO (First In, First Out)**, like a line of people. Elements are added at the **tail** and removed from the **head**. Main implementations are `LinkedList`, `ArrayDeque` and `PriorityQueue` (which serves the smallest element first).

```
Queue<String> queue = new ArrayDeque<>();
queue.offer("First");
queue.offer("Second");
System.out.println(queue.poll());    // First
System.out.println(queue.poll());    // Second
```

25. What is the difference between add() and offer() in a Queue?

Both insert at the tail. The difference appears only in a **capacity-restricted** queue that is full. `add()` throws `IllegalStateException`. `offer()` returns `false`. In `ArrayDeque` or `LinkedList` (no size limit) they behave the same.

26. What is the difference between poll() and remove() in a Queue?

Both remove and return the head. On an **empty** queue, `poll()` returns `null` while `remove()` throws `NoSuchElementException`.

27. What is the difference between peek() and element()?

Both **look at the head without removing it**. On an **empty** queue, `peek()` returns `null` while `element()` throws `NoSuchElementException`.

Task	Returns null when empty	Throws exception when empty
Insert	<code>offer(e)</code>	<code>add(e)</code>
Remove head	<code>poll()</code>	<code>remove()</code>
Look at head	<code>peek()</code>	<code>element()</code>

28. What is a Map in Java?

A Map stores data as **key-value pairs**. Every key is unique and maps to one value. You find a value quickly by its key. Main classes: `HashMap`, `LinkedHashMap`, `TreeMap` and `Hashtable`.

```
Map<Integer, String> students = new HashMap<>();
students.put(101, "Anu");
students.put(102, "Bala");
System.out.println(students.get(101));    // Anu
```

29. What is the difference between a Map and a Set?

A Set stores **single unique elements**. A Map stores **key-value pairs** with unique keys. A Set cannot look up by key. A Map can. Inside Java, a HashSet is actually built on top of a HashMap (the set elements are the map keys).

30. Does a HashMap allow duplicate keys?

No. If you put() an existing key again, the **old value is replaced** and returned. A HashMap allows one null key.

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "A");
map.put(1, "B");           // replaces A
System.out.println(map);   // {1=B}
```

31. Can a HashMap contain duplicate values?

Yes. Only keys must be unique.

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Prabhash");
map.put(103, "Prabhash"); // same value, different key: allowed
System.out.println(map);   // {101=Prabhash, 103=Prabhash}
```

32. What is the difference between HashMap and LinkedHashMap?

Both are fast ($O(1)$). HashMap has **no order**. LinkedHashMap keeps **insertion order** (it also maintains a linked list), with slightly more memory use.

33. What is the difference between HashMap and TreeMap?

	HashMap	TreeMap
Order	None	Sorted by key
Speed	$O(1)$	$O(\log n)$
Null key	One null key allowed	Not allowed
Internal structure	Hash table	Red-Black tree
Extra methods	None	firstKey(), lastKey(), floorKey(), ceilingKey()

34. What is the purpose of keySet(), values(), and entrySet()?

They give **views** of a map: keySet() gives all keys, values() gives all values, and entrySet() gives all key-value pairs. They are mainly used to loop over a map.

```
Map<Integer, String> m = new TreeMap<>();
m.put(101, "Anu");
m.put(102, "Bala");
```

```

System.out.println(m.keySet());    // [101, 102]
System.out.println(m.values());    // [Anu, Bala]
System.out.println(m.entrySet());  // [101=Anu, 102=Bala]

for (Map.Entry<Integer, String> e : m.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}

```

35. What is an Iterator, and why is it used with collections?

An Iterator is an object that lets you **visit the elements of any collection one by one**. Its methods are `hasNext()`, `next()` and `remove()`. It is used because a Set or Queue has no index, so a normal for loop will not work, and because it allows **safe removal** while looping (without `ConcurrentModificationException`).

```

List<String> subjects = new ArrayList<>(List.of("Java", "SQL", "HTML"));
Iterator<String> it = subjects.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("HTML")) {
        it.remove();    // safe removal during traversal
    }
}
System.out.println(subjects);    // [Java, SQL]
ListIterator extends it and can also move backwards (hasPrevious(), previous()).

```

B. Strings in Java (15 Questions)

36. What is a String in Java?

A String is an **object** (class `java.lang.String`) that stores a sequence of characters. It is **final** (cannot be extended) and **immutable** (cannot be changed after creation).

```

String s1 = "Hello";    // string literal
String s2 = new String("Hello");    // using new

```

37. Why is String immutable in Java?

Once created, a String's content cannot change. Any change creates a **new** String. Reasons:

1. Security: values like file paths, URLs and passwords cannot be altered.
2. Thread safety: shared safely between threads, no locking needed.
3. String pool: identical literals can safely share one object.
4. Hash code caching: makes String a fast and reliable HashMap key.

```

String s = "Java";
s.concat(" World");
System.out.println(s);    // Java (unchanged)
s = s.concat(" World");    // new object assigned back to s
System.out.println(s);    // Java World

```

38. What is the difference between String and StringBuilder?

	String	StringBuilder
Mutability	Immutable	Mutable (changes in place)
Speed for many changes	Slow (new object each time)	Fast
Memory in loops	Creates many objects	Uses one object
Main methods	concat(), substring(), replace()	append(), insert(), delete(), reverse()

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
sb.insert(0, ">> ");
System.out.println(sb);    // >> Hello World
```

39. What is the difference between StringBuilder and StringBuffer?

Both are mutable and have the same methods. `StringBuffer` is **synchronized** (thread-safe) but slower. `StringBuilder` is **not synchronized**, so it is faster. Use `StringBuilder` in normal single-threaded code, and `StringBuffer` only when several threads share the same object.

40. What is the difference between == and equals() when comparing Strings?

`==` compares **references** (are they the same object in memory?). `equals()` compares the **content** (are the characters the same?). Always use `equals()` to compare text.

```
String a = "java";
String b = "java";
String c = new String("java");

System.out.println(a == b);    // true  (same object in the pool)
System.out.println(a == c);    // false (c is a new object)
System.out.println(a.equals(c)); // true  (same content)
```

41. What is the String pool?

The String pool is a special memory area inside the heap where **string literals are stored**. When you write the same literal twice, Java reuses one object instead of creating another, which saves memory. A String created with `new` is outside the pool, but `intern()` can move or fetch it from the pool.

```
String a = "hi";                // goes to the pool
String b = "hi";                // reuses the same pool object
String c = new String("hi");     // new object in the heap
String d = c.intern();           // returns the pool object

System.out.println(a == b);     // true
System.out.println(a == c);     // false
System.out.println(a == d);     // true
```

42. What is the use of the length() method?

length() returns the **number of characters** in a String as an **int**. An empty String returns 0. (For arrays, length is a property without brackets.)

```
System.out.println("Hello".length());    // 5
System.out.println("").length());       // 0
```

43. What is the use of the charAt() method?

charAt(index) returns the **character at a position**. The index starts at 0. A wrong index throws **StringIndexOutOfBoundsException**.

```
String s = "Java";
System.out.println(s.charAt(0));    // J
System.out.println(s.charAt(3));    // a
// s.charAt(4) -> StringIndexOutOfBoundsException
```

44. What is the use of the substring() method?

substring() **cuts out a part** of a String. substring(begin) goes to the end. substring(begin, end) includes begin but **excludes** end, so the length is end - begin.

```
String s = "HelloWorld";
System.out.println(s.substring(5));    // World
System.out.println(s.substring(0, 5)); // Hello
System.out.println(s.substring(2, 5)); // llo
```

45. What is the difference between equals() and equalsIgnoreCase()?

equals() is **case-sensitive**. equalsIgnoreCase() ignores upper and lower case.

```
String a = "Java";
System.out.println(a.equals("java"));    // false
System.out.println(a.equalsIgnoreCase("java")); // true
```

46. What is the use of contains()?

contains() checks **whether a String has a given sequence of characters** and returns true or false. It is case-sensitive. It throws **NullPointerException** if the argument is null.

```
String s = "Welcome to Java";
System.out.println(s.contains("Java"));    // true
System.out.println(s.contains("java"));    // false
```

47. What is the use of indexOf() and lastIndexOf()?

indexOf() gives the position of the **first** occurrence. lastIndexOf() gives the position of the **last** occurrence. Both return -1 if nothing is found.

```
String s = "banana";
System.out.println(s.indexOf("a"));    // 1
System.out.println(s.lastIndexOf("a")); // 5
System.out.println(s.indexOf("a", 2)); // 3 (search starts at index 2)
System.out.println(s.indexOf("z"));    // -1
```

48. What is the difference between `replace()` and `replaceAll()`?

`replace()` replaces **plain text** exactly as written. `replaceAll()` takes a **regular expression** (regex), so special characters such as `.` have a special meaning.

```
String s = "a.b.c";
System.out.println(s.replace(".", "-"));      // a-b-c   (the dot is plain text)
System.out.println(s.replaceAll(".", "-"));   // ----- (the dot means any
character)
```

```
String t = "a  b   c";
System.out.println(t.replaceAll("\\s+", " ")); // a b c   (regex: many spaces to
one)
```

`replaceFirst()` works like `replaceAll()` but changes only the first match.

49. What is the use of `trim()`?

`trim()` **removes spaces (and tabs/new lines) from the start and end** of a String. Spaces in the middle are not touched. It returns a new String. Since Java 11, `strip()` does the same but also understands Unicode spaces.

```
String s = "  hello world  ";
System.out.println "[" + s.trim() + "]"); // [hello world]
```

50. How would you reverse a String in Java?

A String has no `reverse()` method, but `StringBuilder` does. Three common ways:

```
String s = "Hello";

// 1. Using StringBuilder (shortest)
String r1 = new StringBuilder(s).reverse().toString(); // olleH

// 2. Using a loop from the end
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) {
    r2 += s.charAt(i);
} // olleH

// 3. Using a char array and two pointers
char[] ch = s.toCharArray();
int left = 0, right = ch.length - 1;
while (left < right) {
    char temp = ch[left];
    ch[left] = ch[right];
    ch[right] = temp;
    left++;
    right--;
}
String r3 = new String(ch); // olleH
```